Napredna Java



Autor: Goran Savić

Interfejsi

- Zbog fleksibilnosti, česta je praksa da programski kod radi sa interfejsima
  - npr. metoda prima interfejs kao parametar
  - moguće je poslati parametar bilo kojeg tipa koji implementira taj interfejs
- Klasična implementacija interfejsa zahteva kreiranje novog fajla i redefinisanje metoda interfejsa

Anonimne klase

- Definisanje nove klase je nepraktično ako se implementacija interfejsa koristi samo na jednom mestu u kodu
- Efikasnije je koristiti anonimne klase
  - konstrukt koji omogućuje istovremenu deklaraciju i instanciranje klase koja implementira interfejs
  - bez potrebe da se definiše nova klasa u posebnom fajlu

Primer anonimne klase

- Ako postoji interfejs

```
public interface Displayable {
      void displayData(String prefix);
}
```

- Kreiranje anonimne klase koja predstavlja implementaciju interfejsa

```
Displayable d = new Displayable() {
      public void displayData(String prefix) {
         System.out.println(prefix + " Test");
      }
};
d.displayData("Ispis je: ");
```

Lambda izrazi

- Uvedeni u Javi 8 kao kraća sintaksa za anonimne klase
- Lambda izraz je implementacija interfejsa
- Može se koristiti samo za interfejse koji imaju tačno jednu metodu
- Ovakve interfejse nazivamo funkcionalni interfejsi
- Primer implementacije interfejsa sa prethodnog slajda

```
Displayable d = pr -> { System.out.println(pr + " Test"); };
```

Lambda izrazi

- Dva sintaksna dela razdvojena znakom ->
- Sa leve strane parametri koje metoda koju implementiramo prima
  - tipovi parametara nisu obavezni
- Sa desne strane implementacija metode interfejsa
  - pošto je funkcionalni interfejs, ima samo tu jednu metodu

Lambda izrazi

- Primer za metodu sa više parametara

```
public interface Displayable {
	void displayData(String prefix, String sufix);
}

Displayable d = (px, sx) -> {System.out.println(px +
    " Novi Sad " + sx);};
```

- Kada ima više od jednog parametra, parametri idu u zagradu
- Zagrade se moraju staviti i kada metoda ne prima parametre

```
Displayable d = () -> { System.out.println("Novi
Sad"); };
```

Lambda izrazi

- Vitičaste zagrade nisu obavezne ako telo metode sadrži samo jedan izraz
- Ako metoda nije void, može da ima return izraz
- Ako je return izraz jedini u telu metode, onda se ne mora navesti reč `return`

```
public interface Verificator {
	boolean verify(int a);
}

Verificator v = number -> number == 0;

boolean verificationResult = v.verify(5);
```

Lambda izrazi

- Sintaksno gledano lambda izraz predstavlja implementaciju interfejsa
- Obzirom da je reč o funkcionalnom interfejsu koji ima tačno jednu apstraktnu metodu, lambda izraz koji predstavlja implementaciju te metode praktično reprezentuje jednu funkciju
- Tako da korišćenje funkcionalnih interfejsa, odnosno lambda izraza omogućuje prosleđivanje funkcija drugim funkcijama
- Lambda izrazima se u Javi realizuju koncepti funkcionalnog programiranja

Java Stream API

- Deo standardne Java biblioteke od verzije 8
- Omogućuje organizovanje podataka u tokove
  - nad podacima iz toka se mogu primenjivati različite operacije korišćenjem lambda izraza
- Tok se može dobiti iz kolekcije metodom *stream()*

Java Stream API - operacije

- forEach
  - za svaki podatak iz toka izvršava definisanu funkciju

```
countries
  .stream()
  .forEach(c -> c.displayCountry());
```

- filter
  - iz toka izdvaja podatke koji zadovoljavaju kriterijum

```
countries
  .stream()
  .filter(c -> c.getPopulation() > 1000)
  .forEach(c -> c.displayCountry());
```

Java Stream API - operacije

- map
  - transformacija podatka iz toka nekom funkcijom

```
countries
  .stream()
  .map(c -> c.getName())
  .forEach(name ->
        System.out.println(name));
```

Java Stream API - operacije

- collect
  - okupljanje podataka iz toka u jedan objekat, npr. u listu

```
countries
  .stream()
  .filter(c -> c.getPopulation() > 10000)
  .collect(Collectors.toList());
```

Java Stream API - operacije

- flatMap
  - za svaki objekat u toku vraća novi *stream* objekat
  - najčešće koristimo da iz objekata u toku preuzimamo njihove 1:N kolekcije kao tok

```
countries
  .stream()
  .flatMap(c -> c.getCities().stream())
  .forEach(city -> System.out.println(
      city.getZipCode() + " " +
      city.getName());
```

Java Stream API - operacije

- Sve operacije možemo podeliti u dve grupe
  1. Međuoperacije
     - rezultuju novim tokom podataka
     - npr. filter, map
     - ne izvršavaju se odmah pri pozivu
     - nego tek pri pozivu terminalne operacije
  2. Terminalne operacije
     - rezultuju konačnom vrednošću
     - npr. forEach, collect
     - tek njihov poziv izvršava lanac operacija
     - lanac operacija se izvršava pojedinačno za svaki element toka (npr. filter – map – forEach)

Java Stream API - operacije

- Pojedinačno procesiranje svakog elementa toka omogućuje paralelizaciju obrade
- Potrebno je kreirati *parallelStream*

```
countries
  .parallelStream()
  .filter(c -> c.getPopulation() > 100000)
  .map(c -> c.getName())
  .forEach(n -> System.out.println(n));
```

Java Stream API – specijalizovani tokovi

- Postoje i specijalizovani tokovi za rad sa podacima tipa int, long i double
  - IntStream, LongStream, DoubleStream
- Omogućuju operacije nad podacima specifične za ovaj tip
  - zbir, prosek, maksimum, minimum

```
int sumPopulation =
    countries
        .stream()
        .mapToInt(c -> c.getPopulation())
        .sum();
```

Standardni funkcionalni interfejsi

- Stream operacije kao parametar primaju funkcionalni interfejs
  - njegova implementacija se može zadati kao lambda izraz
- Standardna Java biblioteka klasa u paketu **`java.util.function`** sadrži određen broj funkcionalnih interfejsa opšte namene
  - metode iz Java Stream API-ja primaju ove interfejse kao parametre
  - programeri mogu ove interfejse da koriste i za svoje metode
  - npr. ako metoda treba da primi funkcionalni interfejs koji prima jedan parametar i ne vraća rezultat
    - ne mora se pisati takav interfejs, jer već postoji - *Consumer*

Standardni funkcionalni interfejsi

Naziv	Opis
Consumer	Operacija koja prima parametar i ne vraća rezultat.
Supplier	Operacija koja ne prima parametre i vraća rezultat.
Function	Operacija koja prima parametar i vraća rezultat
Predicate	Operacija prima parametar i vraća boolean rezultat
BiConsumer	Operacija koja prima dva parametra i ne vraća rezultat
BiFunction	Operacija koja prima dva parametra i vraća rezultat
BiPredicate	Operacija koja prima dva parametra i vraća boolean rezultat
UnaryOperator	Operacija nad jednim parametrom koja vraća rezultat istog tipa kao parametar
BinaryOperator	Operacija nad dva parametra istog tipa koja vraća rezultat istog tipa kao parametri

Method References

- Nekad lambda izraz izvršava samo jednu naredbu koja je poziv neke postojeće metode, npr.
  - `name -> System.out.println(name)`
  - `c -> c.getName()`
  - `c -> c.getPopulation()`
  - `c -> c.displayCountry()`
- U ovakvim slučajevima moguće je referencirati metodu koja će biti pozvana
- To su Method References
  - uvedeno od Jave 8

Method References

- Umesto poziva metode, navodi se naziv metode i klase/objekta nad kojim se poziva

```
countries
  .parallelStream()
  .filter(c -> c.getPopulation() > 10000000)
  .map(Country::getName)
  .forEach(System.out::println);
```

- Može se koristiti i referenca na konstruktor

```
countries
  .stream()
  .map(Country::getName)
  .map(FootballTeam::new)
  .forEach(System.out::println);
```

Upravljanje null vrednostima



Trideset paoka drže osovinu
ali praznina između njih
pokreće kočiju.
(Tao Te King)

Lao Ce (601. p.n.e. – 531. p.n.e.)

Optional

- Podrška za upravljanje referencama koje mogu imati null vrednost
- Koristi se Optional klasa
- Optional je omotač oko proizvoljnog objekta i skladišti
  - referencu na objekat
  - metode za pristup objektu
  - proveru da li je objekat null



Optional<Country>



Optional<Country> kada je Country null

Optional

- Optional kreiramo od proizvoljnog objekta putem
  - Optional.of
    - izaziva izuzetak ako je objekat null
  - Optional.ofNullable
    - ne izaziva izuzetak
- Prazan Optional objekat
  - Optional.empty
- Provera da li je objekat prisutan (nije null)
  - obj.isPresent();

Optional

- Preuzimanje objekta
  - obj.orElse(“Prazno”);
    - vraća sadržani objekat ako nije null
    - ako je null vraća objekat definisan kao argument metode
  - obj.get()
    - vraća sadržani objekat ako nije null
    - izbacuje izuzetak ako je sadržani objekat null

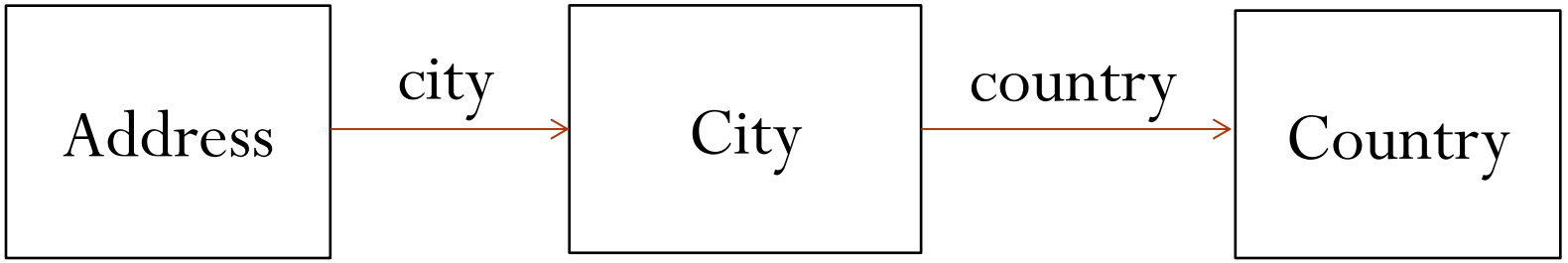
Optional

- Metodom ifPresent se može izvršiti operacija nad objektom ako on nije null

```
Country country = new Country(1, "Serbia",
    7057000);
Optional<Country> c1 = Optional.of(country);
c1.ifPresent(c -> System.out.println(c));
```

- Metodom map može se objekat transformisati ako je prisutan

Optional primer



- Naziv države u kojoj je adresa

```
String name;
if (address != null && address.getCity() != null &&
      address.getCity().getCountry() != null &&
      address.getCity().getCountry().getName() != null) {
   name = address.getCity().getCountry().getName();
} else {
   name = "Nepoznato";
}
```

Optional primer

- Ista funkcionalnost realizovana korišćenjem Optional objekta

```
Optional<Address> a =
      Optional.ofNullable(address);
String name = a.map(Address::getCity)
      .map(City::getCountry)
      .map(Country::getName)
      .orElse("Nepoznato");
```